

SIT225 Task 5C Live Data Plotly Dash Update
For Smartphone Accelerometer Data
Jordan Elkin

Q1

The aim of the implemented solution was to create a system that can continuously receive accelerometer data from Arduino IoT Cloud and update a Plotly Dash visualisation frequently enough that the user experiences the data as a continuous stream. This program is able to handle both continuous data reception and provide dashboard updates while minimising transitions between graph updates.

This solution separates the program into two main areas of responsibility: data acquisition and data visualisation. Accelerometer values are received from Arduino IoT Cloud using the `ArduinoCloudClient`, which is managed through the custom `ArduinoCloud` wrapper class. The Plotly Dash application is run in a separate thread so that receiving new sensor data does not block the dashboard from being updated and vice versa.

The program begins by invoking the `cloud_streaming_graph` wrapper function. This function receives a list containing the Arduino Cloud variable names. For accelerometer data, the cloud variables created in Arduino IoT Cloud are `accel_x`, `accel_y`, and `accel_z`. Rather than manually defining separate functions for each accelerometer axis, the program dynamically creates the required variables, accessor methods, mutator methods, serialisation methods, and callback functions at runtime. This is achieved using Python functions such as `setattr()`, `getattr()`, and `types.MethodType()`.

Each of the dynamic callback functions is invoked each time Arduino IoT Cloud reports a new value for the respective cloud variable. The callback retrieves the generated mutator method and updates the value stored in the generic cloud object. The mutator method also sets the received flag associated with that variable to indicate that a fresh value has been received.

Moreover, the program uses these received flags to control combined writing of the data. This is only carried out when combined data storage is enabled. Before a combined row is written, the program checks that all variables have received an updated value. If one or more have not been updated, the operation returns without writing a sample. Once updated values are available for all variables, the timestamp and current values are written to the shared CSV data store. The flags are then reset so another row will not be written until all variables have received new data.

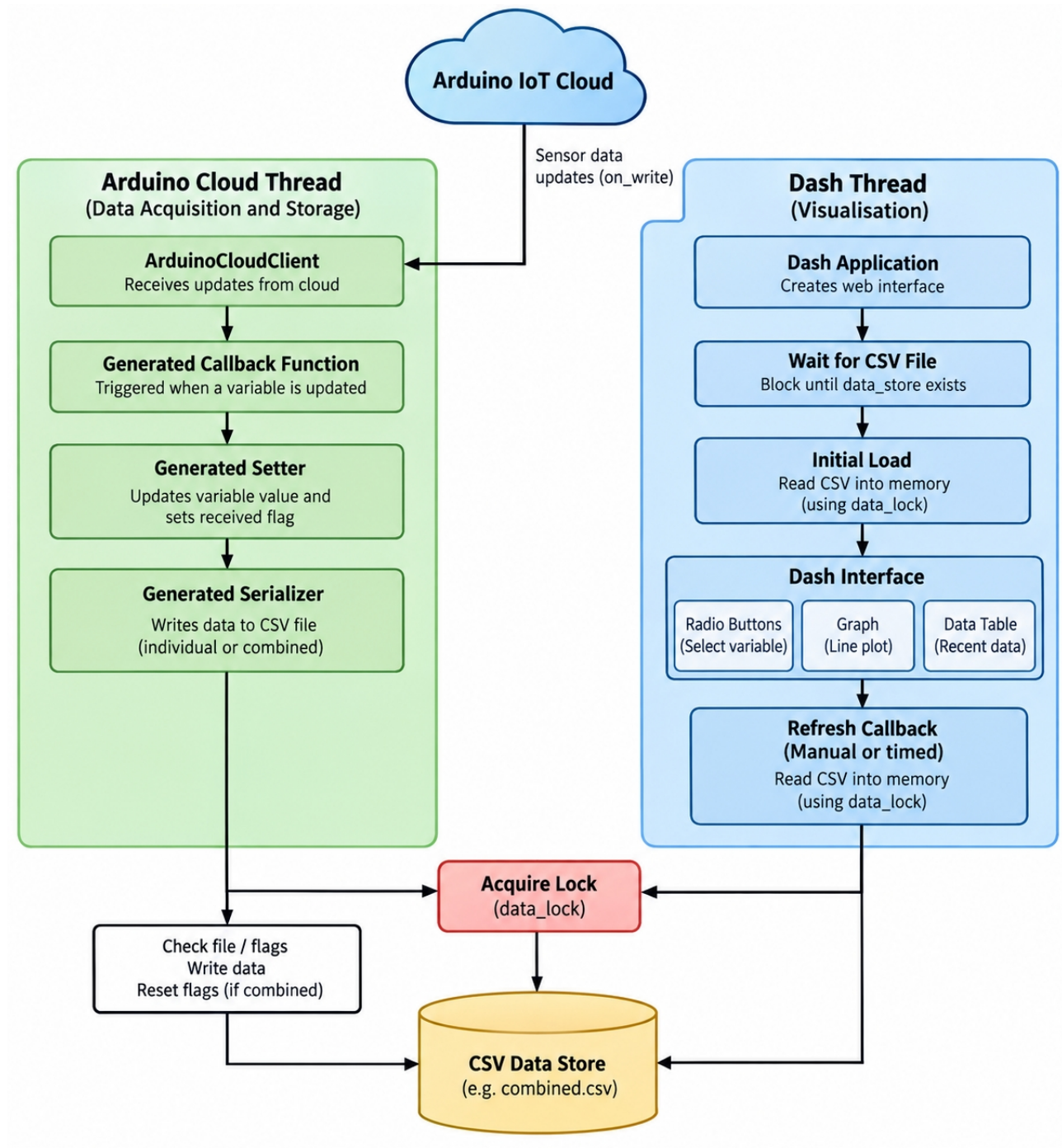
Further, the first complete reading is intentionally not stored when the CSV file is initially created. At this point the program writes only the column headings. This is done in order to discard possible transient or unwanted startup data before the continuous data stream is recorded.

Moreover, because the cloud data acquisition process and Dash interface operate concurrently, both parts of the program may attempt to access the data store at the same time. Before checking or modifying the CSV file, the writer acquires the shared threading lock. The Dash application also acquires the same lock whenever the CSV is read. This ensures that the Dash thread cannot read the file while the Arduino Cloud side is part way through writing a measurement, therefore mitigating the possibility of incomplete data being loaded into the visualisation.

Furthermore, the Dash interface uses a `dcc.Interval` component to trigger automatic data refreshes. The refresh rate is configurable through the `graph_refresh_interval` argument of the wrapper function, with the current default set to 500 ms. When the timer triggers, the `reload_data()` callback acquires the shared lock, reloads the CSV data into a Pandas DataFrame, and stores the updated records inside a Dash `dcc.Store`. This frequent refresh cycle allows newly received samples to be presented without waiting for a fixed number of new readings before the dashboard is updated.

This separates the file refresh from the graph view and enables the data to be streamed from Arduino IoT Cloud to the dashboard. The visualisation is controllable through Dash radio buttons. When the user selects between X, Y, and Z, the graph is redrawn to visualise the selected variable against time. Changing the selected variable does not reload the CSV file; instead, the graph uses the data already stored in `dcc.Store`. This reduces unnecessary disk access and allows the graph view to change immediately while new data continues to be received. Figure 1 shows the complete flow of data from Arduino IoT Cloud through the acquisition and storage process to the Dash visualisation.

Figure 1 - Continuous Arduino IoT Cloud data acquisition and Plotly Dash visualisation flow



Q2

The `cloud_streaming_graph` function was created as a reusable wrapper around the more complex data acquisition, storage, callback generation and Plotly Dash visualisation processes used by the program. Its main purpose is to enable another user to configure continuous Arduino IoT Cloud data streaming without needing to understand the internal implementations such as dynamic callback creation, serialisation, threading, providing updates to Dash or understanding the handling of the data.

The given specification required the wrapper to be reusable for other forms of continuous data and not specific to collecting the accelerometer data from a smartphone. To support this, the wrapper accepts the Arduino IoT Cloud variable names as a list and then dynamically constructs the required object attributes, getters, setters, serialisers, and callback functions for each supplied variable.

The function signature only exposes the configuration values required by the user while the more complex internal behaviour is handled automatically by the wrapper.

Function Signature

```
cloud_streaming_graph(  
    cloud_credentials_file: str,  
    cloud_variables: list,  
    write_combined=False,  
    graph_refresh_interval=500,  
    graph_title=default_title.  
    data_store='combined.csv'  
    sensor_data_name='accelerometer'
```

Parameter Specification

Parameter	Type	Description
cloud_crednetials_file	str	The path to the file containing the Arduino IoT Cloud device credentials used to establish the connection
cloud_variables	list	A list containing the names of the Arduino Cloud variables that will be received, stored, and visualised
write_combined	bool	Determines whether the supplied variables are additionally written into a combined data file after all values are

		received
graph_refresh_interval	int	The automatic dash refresh interval in milliseconds the default value is 500 ms
Graph title	str	The title displayed at the top of the Dash Interface
data_store	str	The CSV file used as the shared data source for the Dash application
sensor_data_name	str	A human-readable name for the sensor or data source used when constructing the plotly graph title

Example Usage

For the smartphone accelerometer used in this task, the wrapper can be called using `cloud_streaming_graph`

```
(
    "key.txt",
    ['accel_x','accel_y','accel_z'],
    write_combined=True
)
```

This call creates the required data-handling components for the three accelerometer axes and starts the Arduino Cloud client and Dash visualisation. The current program creates the cloud variables and associated callbacks by iterating over the supplied cloud variables list.

This same wrapper can also be configured for any other continuous sensor without modification to the internal implementation for example:

```
cloud_streaming_graph(
    "key.txt",
    ['temperature','humidity'],
    write_combined=True,
    graph_refresh_interval=1000,
    graph_title='Environmental Sensor Monitor',
    data_store='environment.csv',
    sensor_data_name='Environmental Sensor'
)
```

The above example demonstrates the intended general-purpose design of the wrapper. The function does not require variable names to hard-coded into the underlying data-handling class. Instead, it generates the require attributes and methods at run-time from the list supplied by the user.

Internal Operation

When `cloud_streaming_graph` is called, the function first creates a `GenericCloudObject` that is used as the runtime container for the supplied cloud variables. For each variable, the program dynamically creates the variable value, a corresponding received flag, a getter, a setter, and serialisation method. These generated methods are attached to the object using `types.MethodType()`. A matching callback function is also generated for each cloud variable. When a new Arduino Cloud value arrives, the callback calls the generated setter and serialiser for that specific variable. If combined writing is enabled, the combined serialiser checks whether fresh data has been received for all configured variables before writing a complete record to the shared CSV file. Once the variable callbacks have been created, they are registered with the Arduino IoT cloud client. The wrapper then starts the Dash interface in a separate thread and runs the cloud client in the main execution flow. This then enables continuous cloud data reception and dashboard visualisation to operate concurrently.

A shared thread lock is used internally to coordinate access to the CSV data store between the arduino cloud data writing operations and the Dash data reading operations. This concurrency handling is abstracted from the user of the wrapper.

Design Justifications

The wrapper was designed to provide abstraction between the end user and the internal data-handling implementation. A user only needs to provide the cloud credentials, variable names and optional graph configuration values. The wrapper automatically performs cloud connection setup, runtime variable creation, callback generation, data serialisation, thread management and Dash visualisation.

This design also improves reusability because the same function can be used for data sources containing different variable names. Instead of implementing a separate application for each sensor type, the wrapper dynamically configures its internal object and callbacks from the supplied variable list.

The function therefore provides a simplified API for continuous Arduino IoT Cloud monitoring while hiding the more sophisticated processing required for dynamic variable handling, callback management, synchronised storage and Plotly Dash updates.

Q3

Link to video presentation showcasing API function usage, please import the entirety of the module in your python file when implementing in your design

<https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=835dc16b-4c3d-4a0a-9bdf-b4bf00514b1f>

Q4

Github Link is below the repo is private please send me a message on teams or on OnTrack for any access issues

<https://github.com/jordanelkin/SIT225/tree/main/Task%205C>

AI Acknowledgement

No AI was used in the creation of this project, traditional sources were used and ideas adapted these are included in the references section.

References

“Builder in Python / Design Patterns.” Refactoring.Guru, 2014, <https://refactoring.guru/design-patterns/builder/python/example>. Accessed 8 Sept. 2026.

“Dependency Injection and Inversion of Control in Python — Dependency Injector 4.39.1 Documentation.” Python-Dependency-Injector.Ets-Labs.Org, https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html. Accessed 8 Sept. 2026.

Hunner, Trey. “Python’s Setattr Function and __Setattr__ Method.” PythonMorsels, Python Morsels, 9 June 2022, <https://www.pythonmorsels.com/python-setattr/>. Accessed 1 Sept. 2026.

Pozo Ramos, Leodanis. “Python Closures: Common Use Cases and Examples.” Realpython, Real Python, 30 Oct. 2024, <https://realpython.com/python-closure/#creating-factory-functions>. Accessed 29 Aug. 2026.

Refactoring Guru. "Factory Method." Refactoring.Guru, 2014, <https://refactoring.guru/design-patterns/factory-method>. Accessed 30 Aug. 2026.